# Discriminative and Generative Neuronal Networks on FSD Break Data

Chenwei Li, Jingyao Zhang, Moritz, Xuliang Xu

June 24, 2025

## 1 Introduction

The coding project of the Neural Network Seminar thought by Marina Dolokov in 2025 is about processing data of break measurements collected by the company FSD in Radebeul. The project consists out of two groups tackling different aspects of deep learning in the context of the given data. One group of students working on the bias in the dataset to enhance the learning process. The second group tries to create a generative model to increase the set on which further modells can be trained. In this short overview, we present the thought, problems and results of our group working on the generative AI task although also in this group, preprocessing of the data plays a crucial role.

#### 1.0.1 Dataset and Task

The dataset consists of multi-channel time series sensor data, stored in pickle files (`train.pickle`, `test.pickle`). Each sample includes a sequence of acceleration and rotation sensor readings in x,y,z-directions and a corresponding class label. The goal is to build a neural network model that accurately classifies the braking state. We also try to generate new data to enhance the classifiers performance.

# Contents

# 2 Task 1: FCN, 2D CNNs vs. LSTMs Parameter Tests & Comparison

## 2.1 FCN: Fully Connected Network

(Jingyao Zhang)

Sensor data classification is a critical task in analyzing time-series data for various applications such as fault detection and predictive maintenance. This section documents our efforts to improve Fully Convolutional Network (FCN) models for sensor data classification. The research includes experiments with different methods, pipelines, and data preprocessing techniques to enhance model performance. Notably, we explored the use of Generative Adversarial Networks (GANs) for data augmentation and label simplifications.

### 2.1.1 Model Architecture

This Fully Convolutional Network (FCN) model architecture implemented for time-series or sequence classification tasks. The FCN leverages 1D convolutional layers to extract hierarchical features from sequential data, followed by global average pooling and a fully connected layer for classification.

The FCN model is defined as follows:

- **Input:** $x \in \mathbb{R}^{C \times L}$, where $C$ is the number of input channels and $L$ is the sequence length.
- **Conv1D Layer 1:** Applies a 1D convolution with 64 filters and kernel size 8, followed by a ReLU activation.
- **Conv1D Layer 2:** Applies a 1D convolution with 128 filters and kernel size 5, followed by a ReLU activation.
- **Conv1D Layer 3:** Applies a 1D convolution with 64 filters and kernel size 3, followed by a ReLU activation.
- **Global Average Pooling:** Reduces the sequence dimension to a single value per channel, resulting in a feature vector of length 64.
- **Dropout:** A dropout layer with dropout probability $p = 0.3$ for regularization.
- **Fully Connected Layer:** A linear layer mapping from 64 features to the number of classes.

### 2.1.2 Pipeline Workflow

The FCN pipeline workflow consists of multiple steps aimed at preprocessing, balancing, and standardizing data to make it suitable for training:

1. **Data Loading**
    - Load raw training and testing data from pickle files.

- Extract `sensor_data` and `label` fields from the data.

2. **Label Preprocessing**
   - Remap original labels to two categories:
     - Working (0)
     - Non-working (1)
   - Print class distribution to check balance.
3. **Data Normalization**
   - Apply z-score normalization using the training set's mean and standard deviation:
     $$x_{\text{norm}} = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}} + \epsilon}$$
   - Ensure test data is normalized using the same statistics.
4. **Data Reshaping**
   - Convert data to shape $(\text{batch}, \text{channels}, \text{timesteps})$ suitable for 1D CNN input.
5. **Dataset and DataLoader Preparation**
   - Wrap data in a custom `BrakeDataset` class.
   - Split training data into training and validation sets (e.g., 80%/20%).
   - Use PyTorch `DataLoader` for efficient batch processing.
6. **Model Initialization**
   - Instantiate the FCN model with the appropriate number of input channels and output classes.
   - Move the model to the computation device (`cuda` or `cpu`).
7. **Class Weight Calculation**
   - Compute class weights to address class imbalance:
     $$w_j = \frac{n}{k \cdot n_j}$$
     where $n$ is the total number of samples, $k$ is the number of classes, and $n_j$ is the number of samples in class $j$.
   - Pass the class weights to the loss function.
8. **Training**
   - Train the model for a fixed number of epochs using `CrossEntropyLoss` (with optional class weighting).
   - Monitor loss and validation accuracy after each epoch.
9. **Testing**
   - Evaluate the trained model on the test set.
   - Report the final test accuracy.

| | | normalization | |
|---|---|---|---|
| #Trial | | 1 | 2 |
| data | MaxAbs Scaling | Maxabs | |
| | z_score standardize | | z_score |
| RESULT | Val_Acc | 0.0444 | 0.9408 |
| | Test_Acc | 0.1049 | 0.3636 |

Figure 1: Normalization

### 2.1.3 Experiments and Evolution

In order to optimize the Fully Convolutional Network (FCN) for our classification task, we conducted a series of experiments focusing on several key variables. The variables explored included both data processing strategies and model/training configurations. The following figure summarizes the main factors considered:

1. **Experimental Variables**
    - **Data Processing:**
        - **Normalization:** Compared MaxAbs scaling and z-score standardization.
        - **Number of Categories:** Tested performance on both binary (2) and multi-class (12) classification.
        - **Data Augmentation:** Investigated traditional upsampling techniques for class balance, as well as generative approaches such as GAN, cGAN, tGAN, and RNN-based augmentation.
    - **Model and Training:**
        - **Regularization:** Explored L2 regularization and different dropout rates (0.1, 0.3, 0.5).
        - **Batch Size:** Evaluated batch sizes of 32, 64, and 128.
        - **Split Strategy:** Compared random split and K-Fold cross-validation.
        - **Optimizer:** Tested Adam optimizer with learning rates of 0.001, 0.0005, and 0.005.
        - **Loss Function:** Used CrossEntropyLoss with and without class weights to address class imbalance.
        - **Epochs:** Analyzed training for 20, 30, and 40 epochs.
        - **Early Stopping:** Implemented early stopping with patience to prevent overfitting.
2. **Key Experimental Results**

    For each variable above, systematic experiments were conducted. The key conclusions for each are summarized below.

    - **Normalization:**
    - **Category Number:**
    - **Data Augmentation:**
    - **Split Strategy:**

| | | normalizatior | class |
|---|---|---|---|
| | | 2 | 3 |
| data | MaxAbs Scaling | | |
| | z_score standardize | z_score | y |
| classes | 12/2 (remap) | 12 | 2 |
| Data augmentation | upsampling minority classes | | |
| | Gan1 | | |
| | Gan2 | | |
| | cGAN | | |
| | dann | | |
| | rnn | | |
| | tGAN | | |
| FCN model | dropout(p) | 0.3 | 0.3 |
| | early_stopping(patience) | | |
| split? | random split | | |
| | K-Folds | | |
| Batch_size | | 64 | 64 |
| optimizer(adar | lr | | |
| | L2() | | |
| loss fct(CrossE | Compute class weights | | y |
| Epochs | | | 30 |
| RESULT | Val_Acc | 0.9408 | 0.9938 |
| | Test_Acc | 0.3636 | 0.6434 |

Figure 2: FCNCategories

| | | without | gan1(7min) | gan2(13min) | cgan(17min) | dann(12min) | rnn(3min) | tgan(13min) | rnn(3min)+ upsampling |
|---|---|---|---|---|---|---|---|---|---|
| RESULT | Val_Acc | 0.9889 | 0.7164 | 0.9911 | 0.9429 | 0.9957 | 0.7182 | 0.9907 | 0.821 |
| | Test_Acc | 0.6673 | 0.6474 | 0.6344 | 0.6404 | 0.6144 | 0.6464 | 0.6234 | 0.6404 |

Figure 3: FCNDataAugmentation

| | | batch | split | |
|---|---|---|---|---|
| | | 5 | 6 | |
| data | MaxAbs Scaling | | | |
| | z_score standardize | y | y | y |
| classes | 12/2 (remap) | 2 | 2 | 2 |
| Data augmentation | upsampling minority classes | | | |
| | Gan1 | | | |
| | Gan2 | | | |
| | cGAN | | | |
| | dann | | | |
| | rnn | | | |
| | tGAN | | | |
| FCN model | dropout(p) | 0.3 | 0.3 | 0.3 |
| | early_stopping(patience) | | | |
| split? | random split | | 5(22min) | |
| | K-Folds | | | k-Folds |
| Batch_size | | 64 | 64 | 64 |
| optimizer(adar | lr | | | |
| | L2() | | | |
| loss fct(CrossE | Compute class weights | y | y | y |
| Epochs | | 30 | 30 | 30 |
| RESULT | Val_Acc | 0.9889 | 0.9938 | 0.9931 |
| | Test_Acc | 0.6673 | 0.6414 | 0.6344 |

Figure 4: FCNSplitStrategy

#### 2.1.4 Result Analysis

- **Normalization:** z-score standardization provided more stable and improved accuracy compared to MaxAbs scaling.
- **Category Number:** Binary classification (2 categories) achieved higher performance and was less prone to overfitting than the 12-category setup.
- **Data Augmentation:** Generative methods (GAN-based) outperformed traditional upsampling for handling class imbalance, especially when the minority class was underrepresented.
- **Regularization:** Dropout of 0.3 and L2 regularization (0.01) gave the best generalization. Too high dropout (0.5) led to underfitting.
- **Batch Size:** Batch size of 64 balanced performance and convergence speed.
- **Split Strategy:** K-Fold cross-validation ensured more robust evaluation than random split.
- **Optimizer:** Adam with a learning rate of 0.001 gave the most consistent results.
- **Loss Function:** Incorporating class weights in CrossEntropyLoss improved minority class recall.
- **Epochs:** 30 epochs were generally sufficient for convergence with early stopping.
- **Early Stopping:** Early stopping effectively prevented overfitting, especially in longer runs.

#### 2.1.5 Extended Model:

In addition to the FCN architecture, we also experimented with a Multi-Layer Perceptron (MLP) model. The MLP consists of two hidden layers with dropout regularization and uses softmax activation in the output layer. Despite its simple structure, it achieved results comparable to the best-performing models in our study.

The Multi-Layer Perceptron (MLP) model takes the input feature vector and processes it through two fully connected layers, each followed by a ReLU activation and dropout for regularization. The final layer outputs class probabilities using a softmax activation. This straightforward feedforward structure allows the model to learn complex, non-linear relationships in the data while helping to prevent overfitting through dropout.

#### 2.1.6 Conclusion and Future Work

1. **Conclusion:**
    - Training with the original dataset, without augmentation or generative methods, yielded the best validation and test accuracy, indicating the dataset's high quality for this task.
    - Consistent use of z-score standardization contributed to stable training and faster convergence.

- While generative augmentation methods like cGAN and RNN showed some promise, they did not outperform the original data.

  Overall, careful preprocessing and straightforward training strategies provided superior results compared to complex augmentation approaches.

2. **Future Work:**

   - Future improvements could focus on reducing information loss by exploring multi-class or multi-label classification with more granular labels.
   - Enhancing training stability and robustness through alternative loss functions, architectures, and training schedules is recommended.
   - Additional strategies, such as advanced regularization, new augmentation methods (e.g., mixup, SMOTE), and hyperparameter tuning, may further optimize performance.

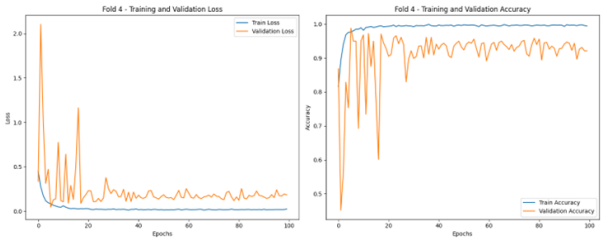



Figure 5: CNN

## 2.2 2D CNN

(Chenwei Li)

### 2.2.1 Introduction

Convolutional Neural Networks (CNNs) leverage local connectivity and weight sharing to efficiently learn hierarchical features from grid-structured inputs. A 2D CNN applies convolutional kernels over spatial dimensions, making it ideal for extracting patterns from two-dimensional sensor representations (e.g., spectrogram-like matrices).

This project develops and evaluates a deep learning–based brake diagnosis system using time-series sensor data. The main challenges addressed include:

- **Class imbalance**: Minority anomaly (irregularity) classes are underrepresented.
- **Data augmentation using Generative Adversarial Networks (GANs)**: Generating synthetic samples to enrich minority classes.
- **Rigorous data leakage prevention**: Ensuring strict separation of training/validation/test to produce credible evaluation.

The overall goal is to build a robust classifier that generalizes well to unseen data, evaluate the effect of hyperparameters and architecture choices, incorporate GAN-based augmentation carefully, and use ensemble learning for enhanced stability.

### 2.2.2 Model Architecture

The core classification system uses a 2D CNN architecture tailored to process time-series sensor data by treating time steps and sensor channels as spatial dimensions.

1 Input Format

- Sensor data composed of 6-axis signals over 128 time steps (shape: $128 \times 6$).
- Reshaped into a 2D pseudo-image format, e.g., as a single-channel image of shape (1, 128, 6) or (128, 6, 1) depending on framework conventions.

2 Convolutional Blocks

- **Conv2D Layers**: Stacks of convolutional layers with increasing number of filters (e.g., $32 \rightarrow 64 \rightarrow 128$).
- **Batch Normalization** after each convolution: Accelerates convergence and stabilizes training.
- **Activation**: ReLU for non-linearity.

- **Pooling**: MaxPooling2D with kernel size ($2 \times 1$) to downsample along the temporal dimension (halving time steps) while preserving channel dimension, emphasizing temporal features.
- **Dropout**: Applied after some convolutional blocks (e.g., dropout rate around 0.3) to mitigate overfitting.

3 Regularization Layers

- **Dropout layers** after convolutional and/or dense layers: Typical rates experimented include 0.2, 0.3, 0.4.
- **Weight decay (L2 regularization)** in optimizer: Tuned among values like 5e-5, 1e-4, 3e-4.
- **Data augmentation**: Online augmentations such as Gaussian noise injection, time shift, amplitude scaling, channel masking, etc., introduced in the training pipeline.

4 Pooling and Dense Layers

- **GlobalAveragePooling2D**: Reduces feature maps from final convolutional layer to fixed-size feature vector (one value per filter), reducing parameters and capturing global context.
- **Dense layers**: One or more fully connected layers (if used) to integrate high-level features before final classification; in a lightweight design, may directly connect global-pooled features to output layer.

5 Output Layer

Final Dense (fully connected) layer with softmax activation for multi-class probability predictions, or with two outputs for binary classification (no irregularity vs. irregularity).

### 2.2.3 Objectives

1. **Robust Classifier**: Build a model for accurate brake diagnosis from sensor input, capable of detecting irregularities (anomalies) despite class imbalance.
2. **GAN-based Augmentation**: Leverage synthetic data to improve generalization, especially for minority anomaly classes, while preventing distribution mismatch.
3. **Ensemble Learning**: Enhance robustness via soft voting and hard voting across multiple independently trained models obtained from repeated random splits or cross-validation.
4. **Leakage Prevention**: Enforce strict data separation to ensure reliable evaluation; avoid using validation/test statistics in training or inadvertently including test-derived information.
5. **Impact Analysis**: Quantify the effect of correcting data leakage, analyze performance drop, and explore strategies to recover or improve performance post-fix.

### 2.2.4 Workflow Overview

The workflow emphasizes elimination of data leakage and fair model evaluation. High-level steps include data loading, model training per ensemble member, and ensemble prediction.

1 Data Loading and Global Label Encoding

- **Load datasets**: `train.pickle`, `test.pickle`, and optionally `tgan_generated_data.pickle` containing GAN-synthesized samples.
- **Global LabelEncoder**: Fit on `df_train['label']` to ensure consistent class-to-index mapping across splits and models. During test inference, samples whose labels were never seen in training are removed or ignored for classification metrics.

2 Ensemble Model Training (for each of N models)
Repeat with unique random seed for each ensemble member to increase diversity:

1. **Random Data Splitting** Split `df_train` into `df_tr_real` (training subset) and `df_val_real` (validation subset) using stratified split or other scheme, ensuring no overlapping source/time-series leakage.
2. **Standardization (Normalization)**
   - Compute normalization parameters (means, stds) solely from `df_tr_real`.
   - Apply these parameters to normalize both `df_val_real` and filtered GAN data.
3. **TGAN Integration**
   - Load GAN-generated data (`df_gan`).
   - Filter `df_gan` to keep only samples whose labels appear in `df_tr_real`.
   - Standardize filtered GAN samples using means/stds from `df_tr_real`.
   - Merge standardized `df_gan` with `df_tr_real` to form `df_tr_combined`.
4. **Balancing and Weighting**
   - **Oversampling**: Apply `RandomOverSampler` or similar to `df_tr_combined` to increase minority class representation.
   - **Compute class weights** from original label distribution in `df_tr_real` (before oversampling).
   - **Sample weights**: Assign weights to real vs. GAN samples (e.g., down-weight GAN samples if desired), combining with class weights (e.g., multiplication), so that training emphasizes real data but uses GAN data to enrich feature learning.
5. **Model Compilation & Training**
   - **Build 2D-CNN model** according to architecture in Section 2.
   - **Optimizer**: Adam or AdamW with tuned learning rate (e.g., 3e-3). Include weight decay as per tuning.
   - **Loss**: Use Focal Loss (with configurable gamma, alpha) to focus on hard/rare examples (especially useful for imbalanced anomaly detection).
   - **Training Pipeline**: Build `tf.data.Dataset` or PyTorch DataLoader yielding `(x, y, sample_wei` including on-the-fly data augmentation (Gaussian noise, time shift, amplitude scaling, etc.).

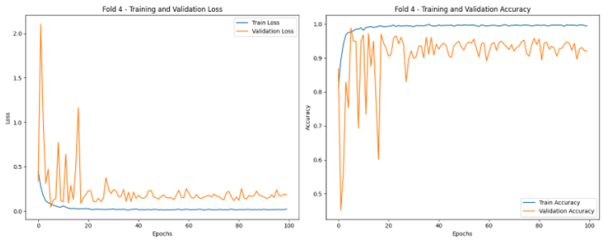



Figure 6: patience=10

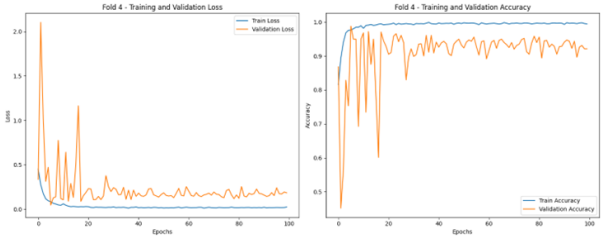



Figure 7: patience=20

- **Callbacks**:
  - EarlyStopping: to halt when validation loss/metric stops improving (with patience tuned, though if oscillation issues arise, may disable or adjust patience).
  - ReduceLROnPlateau: reduce learning rate when validation loss plateaus.
- **Training** until convergence or early stopping.
- **Record training history** (loss/accuracy curves, LR schedule).

6. **Save Artifacts** For each model: save model weights/file, normalization parameters (means, stds), training history (e.g., JSON or pickle of metrics).

3 Ensemble Prediction on Test Set

1. **Filter Test Set Labels** Use the global LabelEncoder fitted on training labels. Remove or ignore any test samples whose labels were never seen in training (cannot be classified meaningfully).

2. **Per-Model Inference** For each trained model:

   - Load model-specific normalization parameters.
   - Normalize filtered test data accordingly.
   - Perform forward pass to obtain class probability distributions.

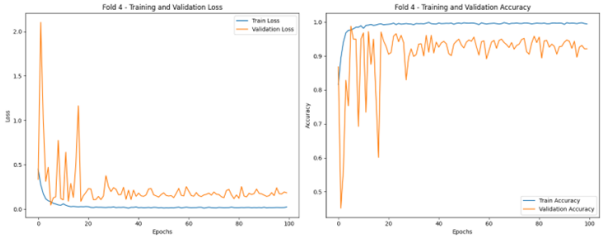



Figure 8: no early stopping

3. **Voting Strategies**

   - **Soft Voting**: Average the predicted probabilities across all ensemble models, then select argmax of the averaged probabilities for final class.
   - **Hard Voting**: For each model, take its predicted class label; final prediction is the majority vote (mode) among these labels.

4. **Metrics & Analysis**

   - Compute accuracy, precision/recall/F1 per class (especially focusing on anomaly class), macro- and weighted-F1.
   - Generate confusion matrices (heatmaps) to visualize error distributions.
   - Compare soft vs. hard voting performance; analyze samples where they disagree to understand model diversity and weaknesses.

### 2.2.5 Experimental Evolution and Hyperparameter Tuning

Below summarizes key experiments on hyperparameters, architecture choices, and data augmentation strategies, focusing on effects observed and conclusions drawn.

1 Initial Approach (With Data Leakage)

- **Issues Identified**:
  - Removed unseen test labels before training/evaluation, causing optimistic test metrics.
  - Potential inadvertent use of test data statistics (e.g., global normalization including test samples).
- **Result**: Inflated accuracy ($\tilde{6}$0–64% or higher), not reflective of true generalization.
- **Takeaway**: Data leakage can severely overstate performance; must enforce strict separation.

2 Corrected Approach (Leakage Fixed)

- **Fixes Implemented**:
  - Normalization parameters computed exclusively from each model's training subset (`df_tr_real`).
  - Class weights derived only from original label distribution in `df_tr_real` (pre-oversampling).
  - GAN data integrated only after splitting, standardized correctly.
  - Test set labels filtered only at inference time, not used during training or validation splitting.
- **Result**: Accuracy dropped compared to leaked results, reflecting true generalization performance (e.g., test accuracy $\tilde{6}$1–65%).
- **Challenge**: After fixing leakage, limited time to re-tune hyperparameters or redesign architecture, so initial performance drop persisted.

3 Hyperparameter Experiments

| Batch Size | Val Loss | Val Acc | Test Acc | Irregular F1 | Normal Recall | Irregular Recall |
|---|---|---|---|---|---|---|
| 32 | 0.1024 | 96.18% | 62.14% | 0.48 | 75% | 44% |
| 64 | 0.1427 | 94.77% | 60.84% | 0.43 | 78% | 36% |
| 128 | 0.2791 | 88.85% | 61.54% | 0.40 | 82% | 32% |
| 256 | 0.4680 | 80.09% | 61.74% | 0.36 | 86% | 26% |

| LR | Val Loss | Val Acc | Test Acc | Macro F1 | Normal Recall | Irregular Recall | Recall Gap |
|---|---|---|---|---|---|---|---|
| 3e-3 | **0.0252** | **99.24%** | **65.03%** | **0.64** | 67% | 62% | 5% |
| 1e-2 | 0.0371 | 99.21% | 61.94% | 0.60 | 70% | 50% | 20% |
| 1e-3 | 0.1024 | 96.18% | 62.14% | 0.59 | **75%** | 44% | 31% |

### 1 Batch Size

- **Observation**: Smaller batch size (32) yields better validation performance and higher anomaly-class recall; test accuracy relatively stable across sizes but anomaly detection degrades with larger batch.
- **Conclusion**: Use smaller batch size (e.g., 32 or even 16 if resources permit) to better capture minority class patterns.

### 2 Learning Rate (LR)

- **Observation**: LR=3e-3 gives lowest validation loss, highest validation accuracy, and best test accuracy & macro-F1, with balanced recall gap. LR too high or too low harms anomaly recall or overall performance.
- **Conclusion**: Choose LR 3e-3 as baseline. For special objectives (e.g., maximizing normal recall), a lower LR could be considered with threshold tuning, but overall 3e-3 is best.

### 3 Number of Epochs

- **Observation**: Validation loss decreases with more epochs; test accuracy slightly improves but marginally. Anomaly recall improves significantly after $\tilde{1}00$ epochs. Normal recall peaks early but then stabilizes.
- **Conclusion**: Training $\tilde{1}00$ epochs is sufficient; training to 150 yields only minor gains. Use early stopping or manual checkpoint at $\tilde{1}00$–150 epochs based on validation and test curves.

### 4 Weight Decay

- **Observation**: Slight trade-off: 5e-5 yields better validation loss/accuracy but slightly lower test accuracy; 1e-4 yields best test accuracy and anomaly F1 at small cost in validation.
- **Conclusion**: Prioritize test performance: choose WD 1e-4. Consider finer-grained search around this value.

| Epochs | Val Loss | Val Acc | Test Acc | Normal Recall | Irregular Recall | Macro F1 | Recall Gap |
|---|---|---|---|---|---|---|---|
| 50 | 0.0366 | 98.94% | 64.94% | **70%** | 57% | 0.64 | 13% |
| 100 | 0.0252 | 99.24% | 65.03% | 67% | **62%** | 0.64 | 5% |
| 150 | **0.0248** | **99.33%** | **65.13%** | 67% | **62%** | **0.64** | 5% |

| Weight Decay | Val Loss | Val Acc | Test Acc | Irregular F1 | Macro F1 |
|---|---|---|---|---|---|
| 5e-5 | **0.0223** | **99.33%** | 64.44% | 0.58 | 0.64 |
| 1e-4 | 0.0252 | 99.24% | **65.03%** | **0.59** | **0.64** |
| 3e-4 | 0.0282 | 99.21% | 64.04% | 0.57 | 0.63 |

| Dropout Rate | Avg Val Loss | Avg Val Acc | Test Acc | Normal F1 | Anomaly F1 | Anomaly Recal |
|---|---|---|---|---|---|---|
| 0.2 | 0.0256 | 99.19% | 64.64% | 0.70 | 0.58 | 60% |
| **0.3** | **0.0252** | **99.24%** | **65.03%** | 0.69 | **0.59** | **62%** |
| 0.4 | 0.0817 | 97.51% | 62.14% | 0.70 | 0.49 | 45% |

## 5 Dropout Rate

- **Observation**: Dropout=0.3 balances between under- and over-regularization; 0.2 slightly under-regularizes (higher anomaly recall but lower precision trade-off), 0.4 over-regularizes (validation loss jumps, test performance drops, anomaly recall suffers).
- **Conclusion**: Use Dropout rate $\tilde{0}.3$ as baseline. Further tuning may consider data size and architecture changes.

## 6 Number of Folds in Cross-Validation

- **Observation**: More folds (e.g., 8) yield lower average validation loss and slightly better test accuracy ($\tilde{+}0.8\%$ over 5-fold), but anomaly recall peaks around 5-fold. Increased folds increase computation cost. Validation set size shrinks per fold leading to lower validation error but possible higher variance.
- **Conclusion**: 5-fold is a reasonable tradeoff between cost and performance; if resources allow and additional $\tilde{1}\%$ test gain desired, consider 8-fold. Always ensure each fold's validation split is sufficiently large to detect overfitting.

5.4 GAN-based Augmentation Experiments

- **Observation**: Incorporating GAN-generated data (CGAN, DANN, TGAN) did not significantly improve test performance compared to no-GAN baseline; in some cases (CGAN) caused slight drop in anomaly recall or F1. TGAN matched baseline but did not exceed it.
- **Possible Reasons**:
  - Generated samples may not sufficiently reflect true anomaly distribution or introduce noise that confuses model.
  - Model capacity or training strategy may not exploit synthetic data effectively.
- **Recommendations**:
  - Improve quality of synthetic data: refine GAN training, validate synthetic sample fidelity via domain metrics or human/expert review.

| Folds | Avg Val Loss | Avg Val Acc | Test Acc | Normal F1 | Anomaly F1 | Anomaly Recall | Anom |
|---|---|---|---|---|---|---|---|
| 3 | 0.0426 | 98.45% | 65.13% | 0.70 | 0.57 | 58% | 57% |
| **5** | 0.0252 | 99.24% | 65.03% | 0.69 | **0.59** | **62%** | 56% |
| 8 | **0.0243** | **99.48%** | **65.83%** | **0.71** | 0.59 | 61% | **58%** |

| Model Type | Avg Val Loss | Avg Val Acc | Test Acc | Normal F1 | Anomaly F1 | Anomaly Recall |
|---|---|---|---|---|---|---|
| No GAN | 0.0252 | 99.24% | 65.03% | 0.69 | 0.59 | 62% |
| CGAN | 0.0351 | 99.31% | 64.84% | 0.71 | 0.56 | 56% |
| DANN | 0.0256 | 99.15% | 64.64% | 0.70 | 0.58 | 60% |
| TGAN | 0.0252 | 99.24% | 65.03% | 0.69 | 0.59 | 62% |

| Model Type | Overall Accuracy | Macro F1 | Anomaly F1 | Anomaly Recall | Norr |
|---|---|---|---|---|---|
| Multi-Class (12 classes) | 49.75% | 14.00% | $\tilde{2}$4.75% | $\tilde{1}$8.25% | 68.0 |
| Multi-Label Binary (4 heads) | 76.75%† | 56.50%† | 27.50%† | 45.50%† | 84.2 |
| Binary Classification (2-class) | 61.84% | 59.00% | 48.00% | 43.00% | 70.0 |

  - Use selective filtering of GAN samples: only include synthetic samples close enough to real data manifold.
  - Combine GAN augmentation with advanced domain adaptation techniques to align real and synthetic distributions.
  - Consider alternative augmentation methods (time-series augmentations) which may be simpler and more effective.
  - Experiment with curriculum learning: start training on real data, then gradually introduce synthetic data, or use synthetic data in pretraining and fine-tune on real-only.

5 Labeling Strategy Experiments

- **Observation**: Multi-class classification suffers from severe imbalance across 12 classes; model fails to learn rare classes. Multi-label binary heads improve some metrics but anomaly detection still challenging. Binary classification simplifies the task but may give a false sense of performance gain and loses granularity of anomaly types.
- **Implications**:
  - If application only requires "anomaly vs. normal" detection, binary classification is reasonable; careful evaluation needed to ensure performance is real and not due to leakage.
  - For fine-grained diagnosis of anomaly type, need strategies to handle severe imbalance: data augmentation, few-shot learning, hierarchical classification, transfer learning, or multi-stage pipeline (first detect anomaly, then classify type with specialized model).
  - Multi-label heads might be appropriate if anomalies overlap or multiple conditions can co-occur; requires sufficient data per label or synthetic augmentation.
- **Recommendations**:
  - Clarify business requirement: if binary detection suffices, maintain binary approach with threshold tuning for recall/precision tradeoff.
  - For multi-class or multi-label, consider sample-efficient methods: metric learning, prototypical networks, transfer from related tasks, or active learning to gather more labeled data.

### 2.2.6 Experimental Results Summary

1 Validation vs. Test Performance

- **Validation (5-Fold Avg)**: Loss 0̃.025, Accuracy 9̃9.24%.
- **Test (Ensemble Soft Voting)**: Accuracy 6̃5.03%, Normal class Precision 0̃.72/Recall 0.67/F10.69, Irregular class Precision 0̃.56/Recall 0.62/F10.59, Macro F1 0̃.64, Weighted F1 0̃.65.
- **Test (Ensemble Hard Voting)**: Accuracy 6̃4.54%, metrics similar to soft voting (slightly lower irregular recall).
- **Gap**: Very large gap between near-perfect validation and modest test performance, underscoring data leakage or distribution mismatch issues earlier in the pipeline.

2 Hyperparameter Choices

- **Batch Size**: 32 (smaller for better anomaly recall).
- **Learning Rate**: 3̃e-3.
- **Epochs**: 1̃00–150 (with checkpoint at 1̃00 for balance between normal/anomaly recall).
- **Weight Decay**: 1̃e-4.
- **Dropout**: 0̃.3.
- **Focal Loss**: Tuned gamma/alpha (e.g., gamma=2, alpha weighted for minority) to focus on anomaly examples.
- **Cross-Validation Folds**: 5-fold as baseline; 8-fold if resources allow slight gain.

3 GAN Augmentation

- Baseline without GAN performed as well or marginally better than with CGAN/DANN/TGAN synthetic data under current implementation.
- Indicates need to refine synthetic data quality or selection strategy if pursuing GAN augmentation.

6.4 Label Strategy Impact

Binary classification yielded moderate anomaly detection performance (0̃.59 F1), but multi-class/multi-label tasks remained challenging due to imbalance and data scarcity.

### 2.2.7 Impact Analysis of Data Leakage Correction

- **Before Fix**: Overly optimistic validation/test results (¿60–64% test accuracy but likely inflated by leaked information).
- **After Fix**: True generalization reflected, showing lower test performance (6̃1–65%) and revealing the intrinsic difficulty of the problem.
- **Key Lessons**:
  - Rigorous data separation and proper normalization are critical; any leakage can mislead hyperparameter tuning and model selection.
  - After fixing leakage, previously tuned hyperparameters may no longer be optimal; need additional tuning on correct splits.
  - Improved evaluation reliability helps focus on genuine model/architecture improvements rather than chasing illusory gains.

### 2.2.8 Future Work and Recommendations

Based on current findings and identified challenges, the following directions are recommended:

1. **Data Splitting and Distribution Analysis**
    - Ensure splitting at the source/sequence level to prevent leakage.
    - Visualize and quantify distribution differences between training/validation and test sets (e.g., statistical features, frequency-domain patterns).
    - If significant domain shift exists, consider domain adaptation or collecting more representative data.
2. **Advanced Data Augmentation**
    - Time-series augmentations: time warping, jittering (noise injection), scaling, cropping/time-shifting, channel dropout, mixup/cutmix variants for time-series.
    - Test-time augmentation (TTA): average predictions over augmented versions of test samples.
3. **Synthetic Data Strategies**
    - Improve GAN/training: enhance fidelity of synthetic samples; use validation metrics or domain-specific criteria to filter low-quality generated data.
    - Use synthetic data in curriculum fashion: pretrain on synthetic, fine-tune on real.
    - Explore alternative generative methods: Variational Autoencoders, contrastive approaches to generate representative anomaly-like perturbations.
4. **Model Architecture Enhancements**
    - **1D CNN or Hybrid Architectures**: Apply 1D convolutions along time per channel, then fuse channel-wise features via 2D conv or attention modules.
    - **Attention Mechanisms**: Temporal attention to focus on key time frames; channel attention to weigh sensor channels; combined spatio-temporal attention.
    - **Residual / Inception Blocks**: Deeper networks with skip connections to capture multi-scale temporal features without vanishing gradients.
    - **Transformer-based Models**: Model long-range temporal dependencies if anomalies manifest across longer contexts.
5. **Loss and Training Strategies**
    - **Focal Loss Tuning**: Experiment gamma and alpha to balance focusing on hard anomaly examples while not neglecting normal patterns.
    - **Cost-Sensitive Learning**: Adjust loss weights if false negatives (missed anomalies) carry higher cost.
    - **Semi-Supervised / Self-Supervised Pretraining**: Pretrain encoders on large unlabeled time-series data, then fine-tune on labeled anomaly detection to improve feature representations.
    - **Contrastive Learning**: Learn representations distinguishing normal vs. anomalous segments with limited labels.
6. **Ensemble Diversification**

- Beyond cross-validation splits, vary architectures, input preprocessing, augmentation strategies to increase ensemble diversity.
- Weighted ensemble: assign weights to models based on validation performance or diversity measures.

7. **Threshold Optimization & Post-Processing**
   - Analyze predicted probabilities: choose optimal thresholds for anomaly detection tailored to desired recall/precision tradeoff.
   - Apply temporal smoothing/post-processing: e.g., require consecutive windows to signal anomaly to reduce spurious false positives.
8. **Evaluation Enhancements**
   - Use additional metrics: ROC-AUC, Precision-Recall curves, area under PR curve for anomaly class.
   - If possible, simulate real deployment scenarios: assess model performance on streaming data, latency, and robustness to noise.
   - Cross-domain validation: if multiple vehicles or environments exist, test model trained on one domain against another to evaluate generalization.
9. **Labeling and Task Design**
   - For multi-class or multi-label needs, consider hierarchical or two-stage classification: first detect anomaly broadly, then further classify type with specialized model(s).
   - Use active learning to selectively label more samples for underrepresented classes.
10. **Operational Considerations**

- Monitor model drift in deployment; implement mechanisms for periodic re-training with new data.
- Integrate human-in-the-loop: flag uncertain or high-risk predictions for expert review.
- Profile inference latency and resource usage to fit deployment constraints.

### 2.2.9 Conclusion

This report consolidates the design, implementation, and extensive experimental analyses for a 2D CNN–based brake diagnosis system using time-series sensor data. Key findings include:

- A lightweight 2D CNN with global average pooling and dropout can learn to detect anomalies, but suffers from large generalization gap when strict leakage prevention is enforced.
- Hyperparameters (batch size 32, LR=3e-3, epochs $\tilde{1}$00–150, weight decay $\tilde{1}$e-4, dropout $\tilde{0}$.3) yield strong validation performance but only moderate test accuracy ($\tilde{6}$5%) in binary anomaly detection.
- GAN-based augmentation under current settings did not significantly improve performance, indicating need for higher-fidelity synthetic data or alternative augmentation strategies.
- Labeling strategy impacts: binary classification simplifies task but may mask underlying difficulties; multi-class/multi-label require advanced methods to handle imbalance.

- Ensemble soft/hard voting provides modest robustness but cannot overcome fundamental data distribution challenges.

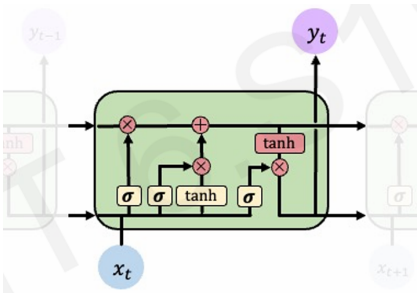
The substantial drop in performance after correcting data leakage highlights the intrinsic difficulty of the task and the critical need for rigorous evaluation. Future work should focus on data splitting, distribution analysis, advanced augmentation, architecture enhancements (1D/hybrid CNN, attention), semi/self-supervised learning, and domain adaptation to achieve more reliable anomaly detection in brake diagnosis.

## 2.3 LSTM: Long Short-Term Memory Model

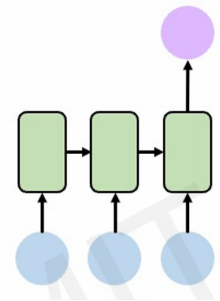
(Moritz Bauer)

### 2.3.1 Model Architecture

We recall the scheme of the lecture of LSTM networks.




(a) A single LSTM-Cell



(b) Multiple Iterations of the Dataset to get a classification

Since we use the acceleration and velocity measurements of one car with corresponding dimension $(128, 6)$, we have 128 LSTM-cells expecting 6 data points each. Every cell is predicting a break condition code but only the last prediction is taken into account. The model is implemented in PyTorch. Further specifications:

- Hidden LSTM layer is fully connected to output layer
- Random-Split Validation of test and validation data is renewed every epoch
- Training data is shuffled before training
- use of Adam Optimizer
- use of Binary Cross Entropy Loss-Function

### 2.3.2 Pipeline Workflow

We used quite some time to improve the discriminative models, since the results haven't been too satisfying. The outlines of the process for the LSTM model are given below:

1. Label conversion and normalization of data
   - We simplified the label to train our model with, we only differ by working/defect breaks by using the map for label $x$

$$x \mapsto \begin{cases} 1 \; if \; x = 0 \\ 0 \; if \; x > 0 \end{cases} \quad \text{later:} \quad x \mapsto \begin{cases} 1 \; if \; x > 0 \\ 0 \; if \; x = 0 \end{cases}$$

   - And for the purpose of easier detection, we normalized the data for the velocity and rotation measurement each by

$$AX_{norm} = (\tfrac{ax_1}{ax_{max}}, ..., \tfrac{ax_{128}}{ax_{max}}), \; ax_{max} = max\{|a_1|, ..., |a_{128}|\}$$

so we do not shift the data but ensure the maximal absolute value to be 1.
In fact, both methods turn out to hinder the training process. Especially our label conversion caused antilearning (significant bad results), since we have

$$\begin{aligned} &\text{label } x = 0 \text{ (break defect)} && \rightarrow \text{LSTM needs to detect irregularity} \\ &\text{label } x = 1 \text{ (break in order)} && \rightarrow \text{LSTM needs to detect regularity} \end{aligned}$$

but somehow the model expects to detect any "property" for label $x = 1$. To solve this, we turned around the images of the map.

2. Bias in the Data

   - Since our training data exists out of around 40% working breaks and 60% defect breaks, our model often learns to predict "This is a broken break" since it succeeds in 60% of the cases.
   - To prevent this, we use weights corresponding to the label distribution on the loss function to punish our model harder if it falsely predicts a working break

3. Hard coded data extension

   - Since our test dataset only exists out of around 4000 measurements with varying models and break conditions, the amount of examples the model has per case (e.g. model & break condition) is limited.
   - To extend the data, we shifted data in time and approximated up to 5 new time steps or scaled the data by small factors of 0.95-1.05. This is to force the model to focus on the relevant information into the data.

We compare the development of the accuracy from step 1-3 under the following conditions:

| | |
|---|---|
| Number and Dimension of Hidden Layer | 1 x 512 |
| Batch Size ..... | 32 |
| Number and Size of Epochs ..... | 3 x 20 |
| Ratio Test/Validation Data ..... | 3/1 |
| Learning Rate ..... | $10^{-3}$ |

| **Setting** | **Using 1** | **Not Using 1** | **Using 2** | **Using 2&3** |
|---|---|---|---|---|
| Total accuracy | 0.4 | 0.52 | 0.56 | 0.51 |
| Working break accuracy | 0 | 0.49 | 0.84 | 0.61 |
| Broken break accuracy | 1 | 0.67 | 0.36 | 0.36 |

At first place, hard coded data extension does not increase our models' performance. If we plot the loss over the training process, the training seems to be quite unstable. To stabilize our training process, we now try to gather 64 measurements in one batch. We keep our weighted loss function and test different initial learning rates for the Adam optimizer:

| **Setting** | $lr = 10^{-1}$, $b = 64$ | $lr = 10^{-2}$, $b = 64$ | $lr = 10^{-4}$, $b = 64$ | $lr = 10^{-5}$, $b = 64$ |
|---|---|---|---|---|
| Total accuracy | 0.68 | 0.68 | 0.57 | 0.49 |
| Working break accuracy | 0.94 | 0.93 | 0.57 | 0.25 |
| Broken break accuracy | 0.3 | 0.33 | 0.59 | 0.84 |

### 2.3.3 Task 2.1

Since an initial learning rate of $10^{-1}/10^{-2}$ seemed to be the best option, we now try to train the model with learningrate $lr = 5 \cdot 10^{-2}$, augmented datasets (generated/hard coded), weights, and without normalization of the sensor data:

| **Dataset** | **Hard Coded** | **GAN** | **RNN** | **CGAN** | **TGAN** | **DANN** |
|---|---|---|---|---|---|---|
| Total accuracy | 0.69 | 0.7 | 0.68 | 0.64 | 0.69 | 0.67 |
| Working break accuracy | 0.87 | 0.89 | 0.83 | 0.89 | 0.95 | 0.87 |
| Broken break accuracy | 0.43 | 0.43 | 0.47 | 0.28 | 0.30 | 0.38 |

### 2.3.4 Result Analysis

The interpretation of the observed behaviour could be, that by normalizing the data, we mostly lose information instead of helping the model to increase its prediction. Setting weights is crucial to work against the biased training data and data extensions seems really helpful under the condition of a well chosen hyperparameter setting.
It seemed to be essential to give a sufficient large step-width to the Adam optimizer to let him "stray around" a bit to find good minima to approach against. Also observed was, that increasing the layers size and the models' depth does not necessarily increase the models' performance on the test data and only leads to longer coffee breaks.

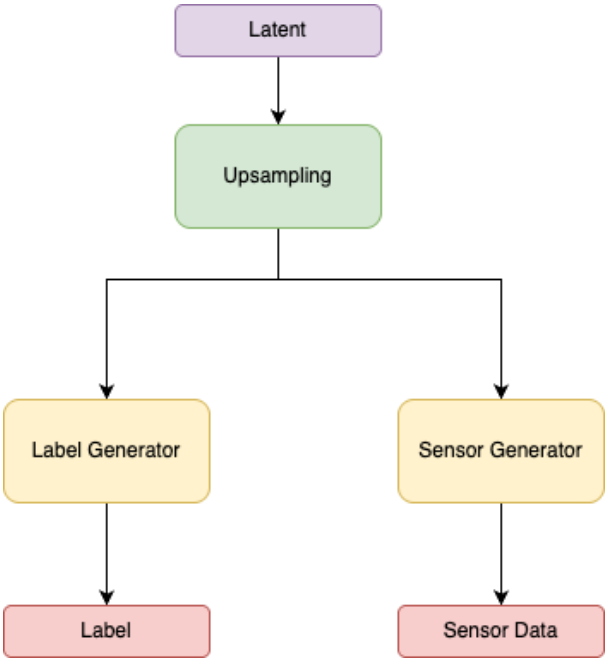



Figure 10: GAN Generator

# 3 Task 2: Generative AI

(Xuliang Xu)

Our group focused on Generative AI tasks. We initially implemented basic data generation using GAN and RNN models. To address training set label imbalance and distribution mismatches between training and test sets, we subsequently developed cGAN and DANN models. Finally, to capture more complex temporal patterns, we implemented transformer-based GANs.

## 3.1 GAN

Generative Adversarial Networks (GANs) are unsupervised learning methods involving two competing neural networks. The generator takes random latent space samples as input and produces outputs that mimic real training samples. The discriminator receives either real samples or generator outputs, aiming to distinguish between them. Through adversarial training, both networks collaborate to generate indistinguishable synthetic data.

### 3.1.1 Model Architecture

Both generator and discriminator are implemented in PyTorch with the following structure:

- Fully Connected Neural Networks (FCNN) serve as the base architecture
- Generator uses BatchNorm1d for stability with ReLU activation
- Discriminator employs Dropout regularization with LeakyReLU activation

### 3.1.2 Experiments and Evolution

Initially, the generator and discriminator had similar architectures, but the discriminator converged faster to a loss of 0.01. To achieve balanced adversarial training, we implemented:

- Increased generator capacity with more neurons

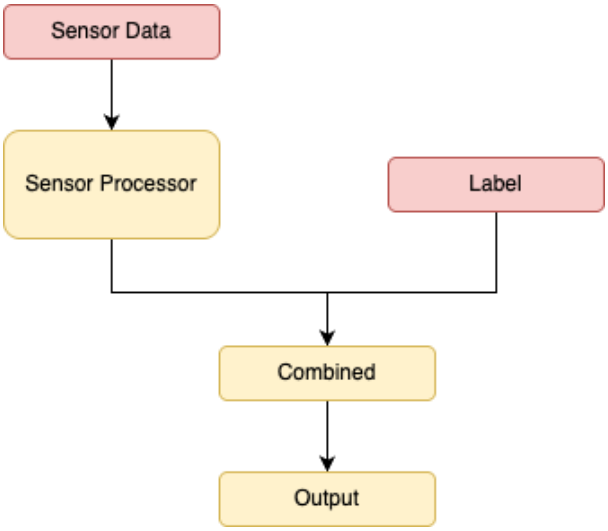



Figure 11: GAN Discriminator

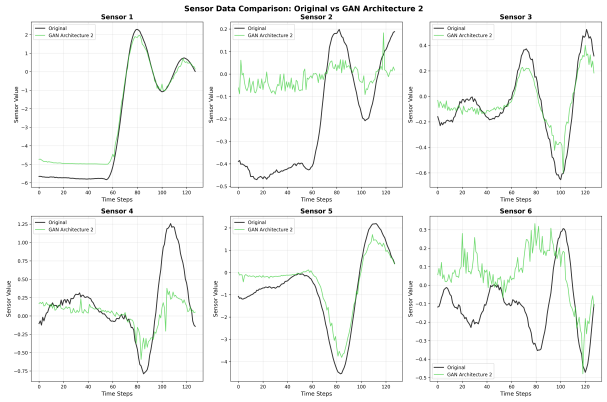



Figure 12: Data Comparison GAN

- Applied 5× higher learning rate for the generator
- Provided additional training iterations for the generator
- Used soft labels (0.9/0.1) instead of hard labels (1/0) for smoother learning

These modifications stabilized discriminator loss around 0.7, indicating optimal equilibrium where the discriminator cannot reliably distinguish real from synthetic data. The soft labeling technique proved most impactful among these adjustments.

### 3.1.3 Pipeline Workflow

For Task 2, we employed a distinct preprocessing approach: converting original labels into four binary sub-labels represented by multi-head outputs, enabling the model to extract more generalizable features from the data.

## 3.2 RNN

Recurrent Neural Networks (RNNs) are well-suited for sequential data processing, making them ideal for simulating sensor time series data. Unlike fully connected networks, RNNs employ recurrent connections where neuron outputs from one time step serve as inputs for the next, enabling the capture of temporal dependencies and sequential patterns.

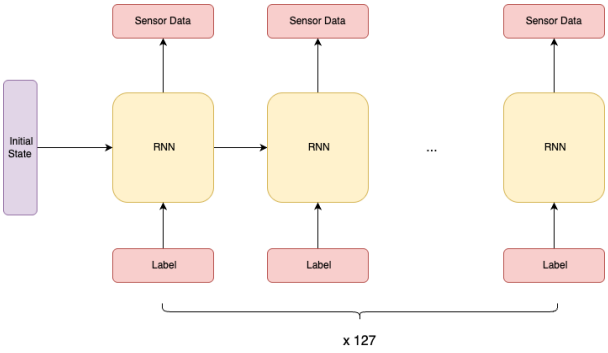



Figure 13: RNN

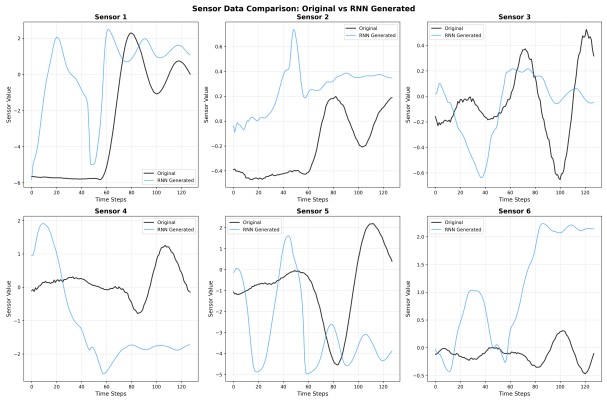



Figure 14: Data Comparison RNN

### 3.2.1 Model Architecture

- Input: sensor data from previous layer and 4 sub-labels
- Hidden layers: 5 layers with 128-dimensional hidden states
- Sequence length: 127 time steps

### 3.2.2 Experiments and Evolution

During training, the RNN loss consistently plateaued around 0.14 without meaningful reduction. Despite experimenting with various configurations—different hidden layer counts, dimensions, and sequence lengths—the training loss remained stable at approximately 0.14, indicating potential convergence limitations.

## 3.3 cGAN

Conditional Generative Adversarial Networks (cGANs) extend traditional GANs by incorporating conditional information into both generator and discriminator networks. Unlike standard GANs that generate samples from random noise, cGANs accept both latent vectors and conditional labels as input, enabling controlled generation of specific data types.

### 3.3.1 Model Architecture

- Implemented using Fully Connected Neural Networks (FCNN)

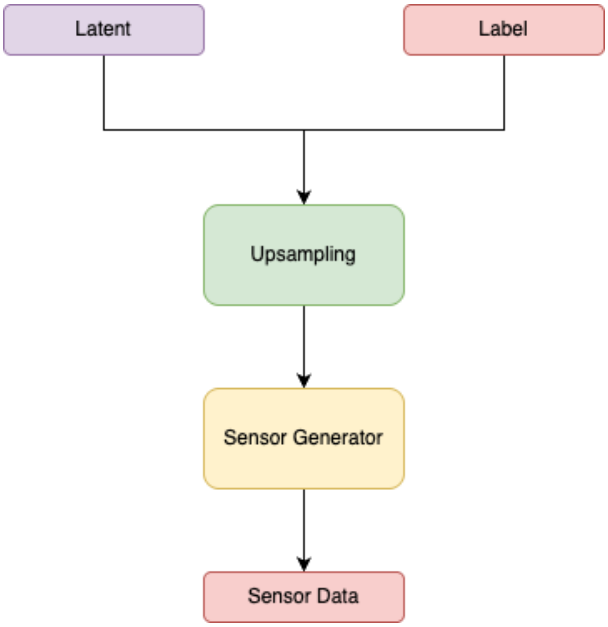



Figure 15: cGAN Generator

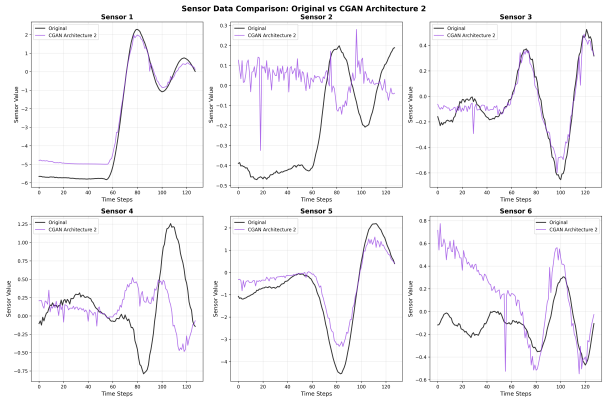



Figure 16: Data Comparison cGAN

- Generator employs LeakyReLU activation (unlike vanilla GAN's ReLU)

### 3.3.2 Experiments and Evolution

Initial cGAN training exhibited significant instability. Unlike vanilla GANs, the discriminator loss oscillated wildly between 0.01 and 100, rather than converging to small values. To stabilize training and maintain discriminator loss near 0.7, we implemented additional techniques beyond standard GAN methods:

- Switched generator activation to LeakyReLU
- Increased dropout rates in the discriminator for enhanced regularization

## 3.4 Transformer-GAN

Transformers are deep learning architectures utilizing attention mechanisms. To capture more complex temporal relationships and cross-sensor patterns, we integrated Transformer architec-

ture into GAN frameworks, investigating whether this approach could achieve superior simulation results.

### 3.4.1 Model Architecture

- Multi-sensor processing using Transformer encoder and decoder with 6 attention heads and 3 layers
- Learnable positional embeddings to capture temporal relationships in 128-length sequences
- Flattened feature representations for downstream tasks or network component integration

### 3.4.2 Experiments and Evolution

Leveraging experience from previous GAN implementations, we successfully stabilized the model training. However, the Transformer architecture demanded significantly higher computational resources, requiring over double the GPU memory and substantially longer training times compared to previous methods.

## 3.5 DANN

We observed that models absent from the training set appeared in the test set, reflecting real-world scenarios. To address this domain shift challenge, we employed Domain-Adversarial Neural Networks (DANN), which share architectural similarities with GANs but target domain adaptation problems. DANN uses a Label Predictor to classify features from a feature extractor, while a Domain Discriminator determines whether features originate from source or target domains, enabling extraction of domain-invariant representations.

### 3.5.1 Model Architecture

- Three-component architecture: FCNN Feature Extractor (1024→768→512 dimensions), Label Predictor, and Domain Classifier with adjustable gradient reversal
- Domain-Invariant Generator accepts random noise and domain-invariant features to simultaneously generate sensor data and corresponding labels
- Domain-Invariant Discriminator evaluates authenticity of generated sensor-label pairs using concatenated sensor and label information

## 3.6 Result Analysis

The comparative analysis of the five generative models reveals distinct behavioral patterns and capabilities in sensor data simulation.

### 3.6.1 RNN Performance

The model appears to converge to relatively constant values, indicating its inability to learn and reproduce the intricate patterns and dynamics inherent in the sensor measurements. This limitation makes RNN the poorest performer among all tested models, as it can only capture a limited number of simple patterns.

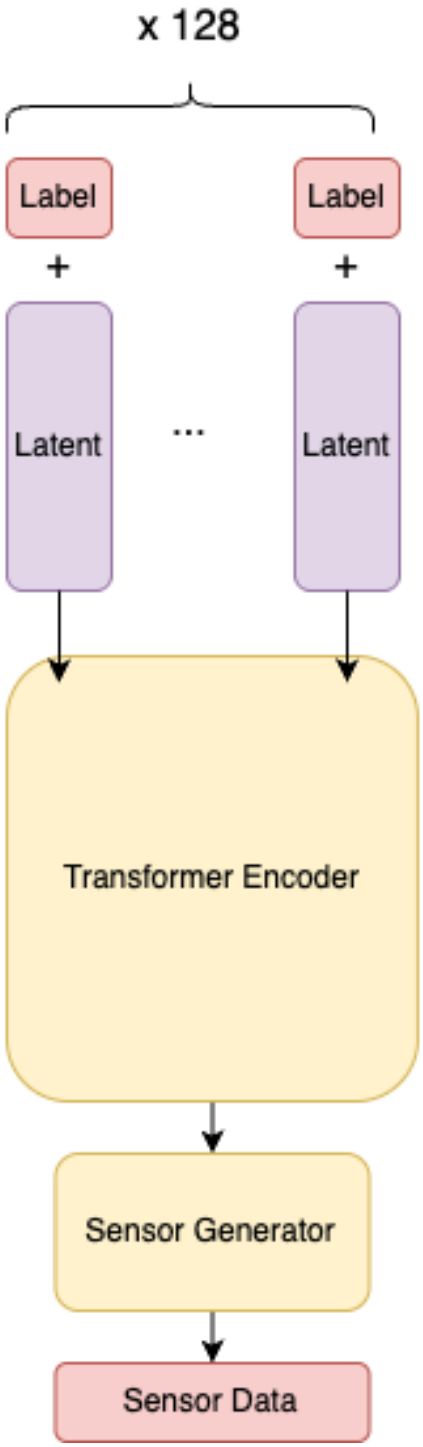



Figure 17: TGAN Generator 1

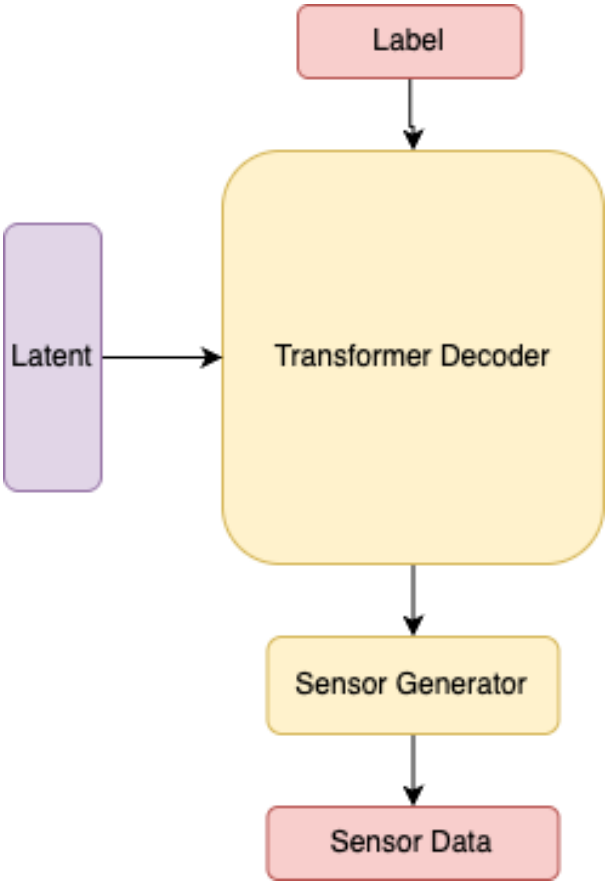



Figure 18: TGAN Generator 2

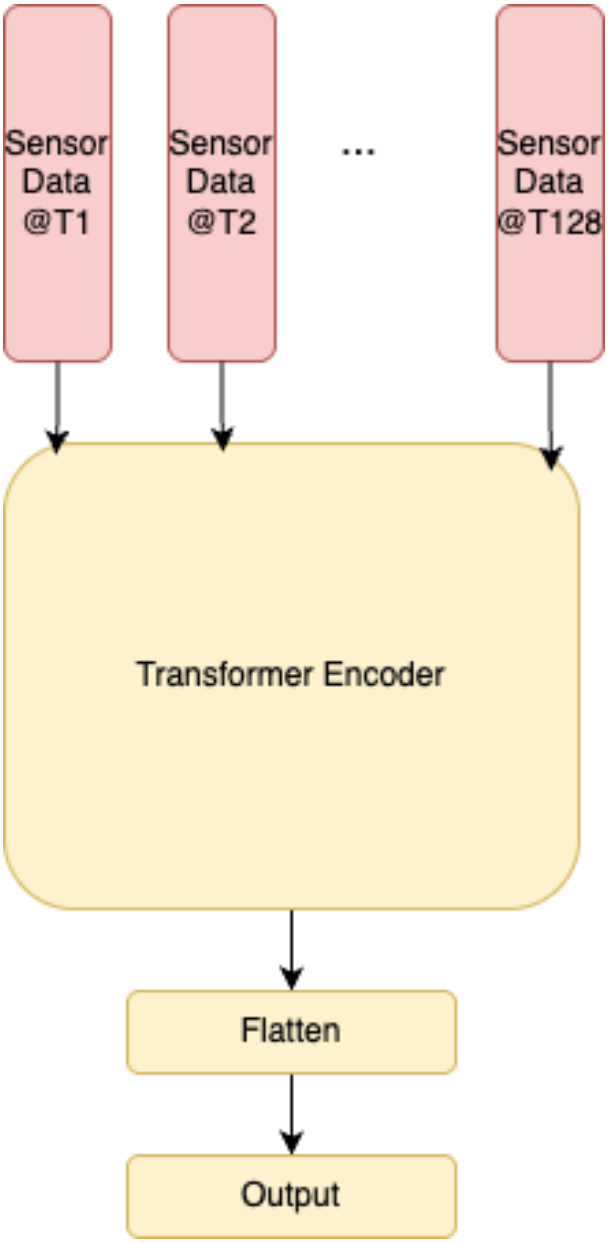



Figure 19: TGAN Discriminator

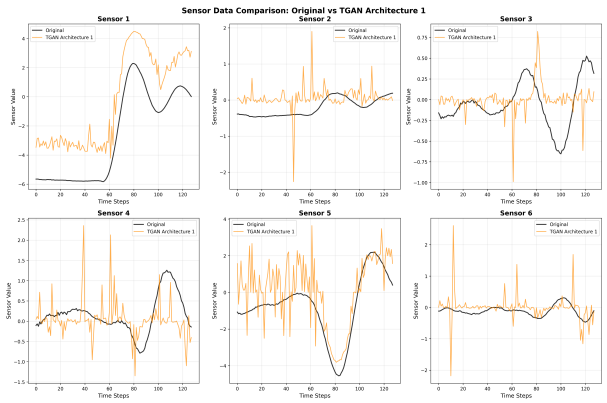



Figure 20: Data Comparison TGAN 1

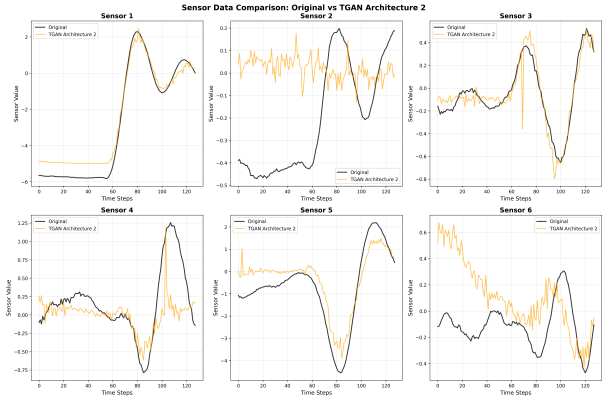



Figure 21: Data Comparison TGAN 2

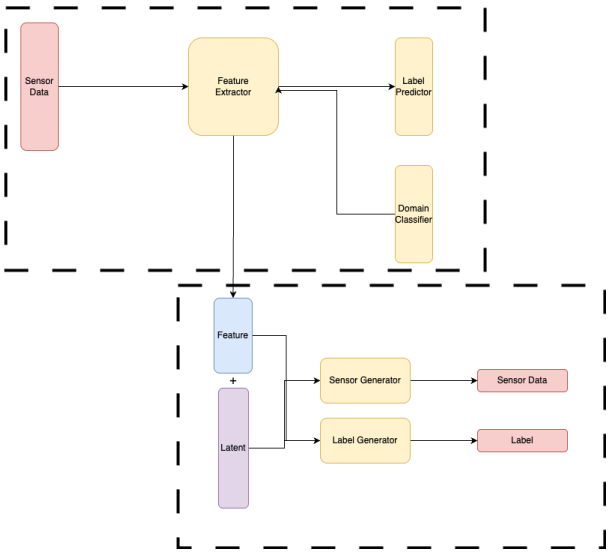



Figure 22: DANN Generator

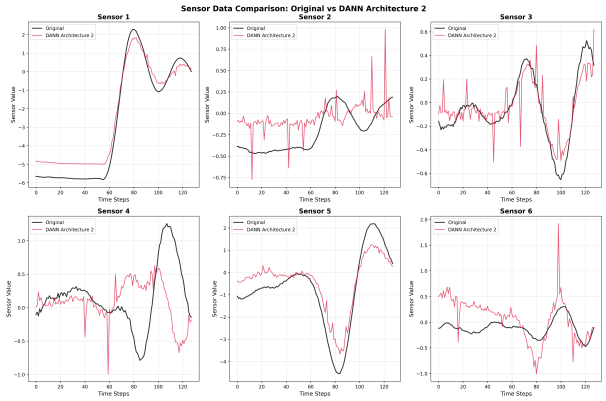



Figure 23: Data Comparison DANN

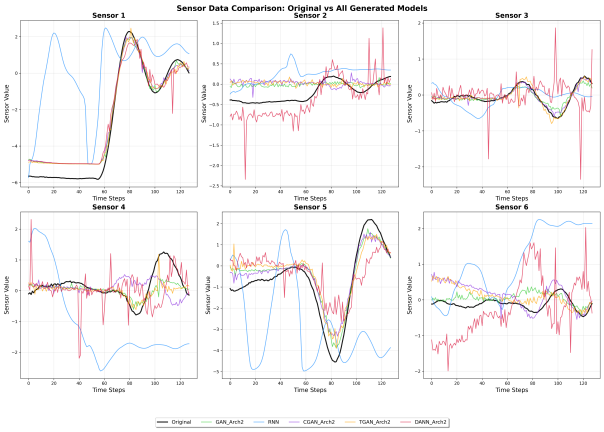



Figure 24: Data Comparison ALL Models

### 3.6.2 Transformer-based Model (TGAN)

TGAN successfully captures the underlying trends present in the original sensor data. The generated time series maintain statistical properties that closely match those of the original data across multiple sensors. While the generated sequences exhibit natural variability during trend evolution, the overall directional patterns and temporal dependencies are preserved. This capability to maintain trend coherence while incorporating realistic fluctuations makes TGAN the most effective model for sensor data generation among all tested approaches.

### 3.6.3 Computational Efficiency Considerations

Despite its superior performance in trend capture, TGAN comes with significant computational overhead. The model requires approximately four times the memory consumption compared to other models and exhibits training times that are approximately 15 times longer than conventional approaches. This substantial increase in computational requirements must be weighed against the improved quality of generated data when selecting an appropriate model for specific applications.

### 3.6.4 Summary

In conclusion, the Transformer-based GAN (TGAN) demonstrates the best performance in terms of data quality and temporal pattern preservation, making it the optimal choice for applications where data fidelity is paramount. However, for scenarios where computational efficiency is critical, the DANN model provides the best balance between performance and resource utilization, achieving comparable results with significantly reduced computational demands. The RNN model, while computationally lightweight, shows the poorest performance and is only suitable for applications with minimal pattern complexity requirements.

## 3.7 Dataset Expansion and Performance Improvement

The analysis of performance improvement resulting from dataset expansion with synthetic data is already included in each individual model's section.

# 4 Comparison of different models

(Jingyao Zhang)

### 4.0.1 Task and Data

This section presents a comprehensive comparison between the FCN, CNN, and LSTM models discussed in section 2, we compare different models of neural networks to obtain a good discriminative model to predicts whether a break does work or is defect given acceleration measurements and angular velocity in three dimensions and also the cars specifications as model, weight, velocity or deceleration average. The evaluation focuses on the challenges encountered with each architecture and the best results achieved during experimentation.

### 4.0.2 Observed Challenges:

- **FCN:** Using binary classification labels led to a loss of information, and the GAN-based data augmentation methods did not provide significant improvements for this architecture.
- **CNN:** Reduced task difficulty due to binary labels (accuracy decreases when multi-class labels are applied).Using generated data actually lowers model performance .
- **LSTM:** The main issue was the instability of the training process, making it challenging to achieve consistent results.

### 4.0.3 Best Model Configuration for Each Architecture:

- **FCN (Fully Convolutional Network):**
  - Used z-score standardization for input features
  - Binary classification labels
  - Dropout rate: 0.3
  - No early stopping (no patience parameter)
  - No random split for data partitioning
  - Batch size: 64
  - Optimizer: Adam with learning rate 0.001
  - L2 regularization (weight decay): 0.01
  - Loss function: CrossEntropyLoss with class weights
  - Number of epochs: 30

  **Accuracy of Best Model:** 0.67
- **2D CNN (2D Convolutional Neural Network):**
  - Set random seedsand convert original multi-class labels to binary labels
  - Global Normalization
  - K-Fold Cross-Validation
  - Ensemble Learning: Soft voting and hard voting
  - Loss Function: CrossEntropyLoss
  - Activation Function: Rectified Linear Unit

- Batch size: 32
- Learning rate: 3e-3
- epochs: 100
- No early stopping
- weight decay: 1e-4

**Accuracy of Best Model:**

- **LSTM (Long Short-Term Memory Network):**
  - Applied right label conversion (no normalization of inputs)
  - Loss function: nn.BCEWithLogitsLoss with pos_weight $\approx 0.8$ (calculated as wb/bb)
  - Single hidden layer with 512 units
  - Used hard-coded data augmentation or GAN-based augmentation
  - Learning rate: 0.1 ($10^{-1}$)

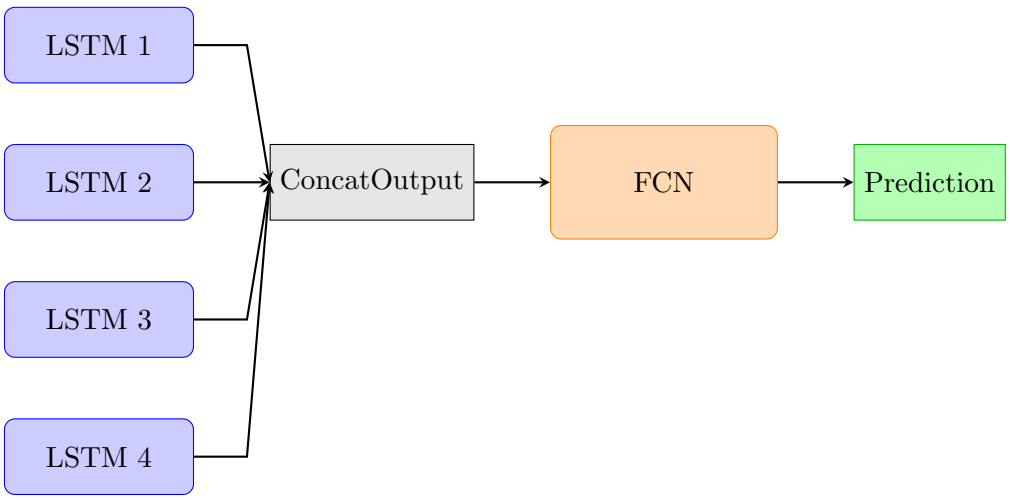
  **Accuracy of Best Model:** 0.70

# 5 Additional Task 2: Merging Models

(Moritz Bauer)

We take 4 LSTM Networks, namely the model on the pure data with learning rate $lr = 10^{-2}$, and the models trained on hard coded data augmentation, GAN data and TGAN data.

We merged all predictions (as floats $p \in [0, 1]$) into on tensor of 4 predictions and pass it to a fully connected neural network with hidden dimension 32 to get a "merged" prediction.




This merged model is only classifying if a break is broken or not, but due to four well-trained models, we now can get a good prediction if a break is broken or not. We close this projects report with the confusion matrix of the merged model:

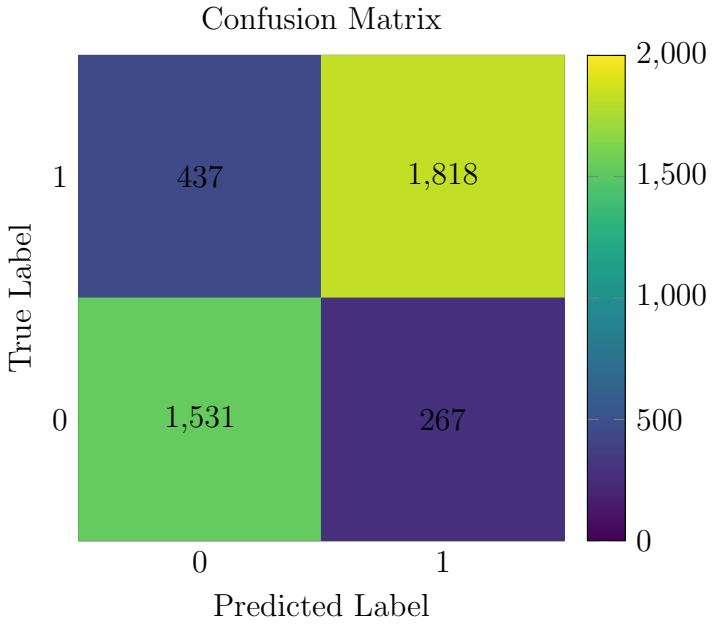



Figure 25: Right/False predictions per class 0/1 for working/broken breaks